

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل پنجم: Index compression

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

5.1 Statistical properties of terms in information retrieval

5.1.1 Heaps' law: Estimating the number of terms

5.1.2 Zipf's law: Modeling the distribution of terms

5.2 Dictionary compression

5.2.1 Dictionary as a string

5.2.2 Blocked storage

5.3 Postings file compression

5.3.1 Variable byte codes

5.3.2 γ codes

5.4 Chapter 5 Summary

فصل ۵: فشرده‌سازی ایندکس - هنر کوچک کردن داده‌ها بدون از دست دادن اطلاعات

تصور کنید مدیر داده گوگل هستید

هر روز میلیاردها جستجو در گوگل انجام می‌شود. برای پاسخ به این جستجوها در کسری از ثانیه، گوگل باید یک ایندکس عظیم از کل وب را مدیریت کند. این ایندکس آنقدر بزرگ است که اگر بخواهیم آن را بدون فشرده‌سازی ذخیره کنیم، به هزاران سرور نیاز داریم! اما گوگل چگونه این کار را انجام می‌دهد؟ پاسخ در فشرده‌سازی هوشمندانه ایندکس‌هاست.

فشرده‌سازی ایندکس یکی از مهم‌ترین تکنیک‌ها در سیستم‌های بازیابی اطلاعات است. در این فصل، با روش‌های فشرده‌سازی دیکشنری و ایندکس معکوس آشنا می‌شویم که برای سیستم‌های بازیابی اطلاعات کارآمد ضروری هستند.

💰 مزایای فشرده‌سازی: فراتر از صرفه‌جویی در فضا

اولین مزیت فشرده‌سازی که به ذهن می‌رسد، کاهش فضای ذخیره‌سازی است. با نسبت فشرده‌سازی 1:4، می‌توانیم هزینه ذخیره‌سازی ایندکس را 75% کاهش دهیم. اما دو مزیت مهم دیگر هم وجود دارد:

۱. **افزایش استفاده از حافظه کش:** تصور کنید در یک کتابخانه دیجیتال بزرگ، کاربران مدام به دنبال کلمه «کرونا» می‌گردند. اگر لیست ارسال این کلمه را در حافظه اصلی ذخیره کنیم، پاسخ به این جستجوها بسیار سریع خواهد بود. با فشرده‌سازی، می‌توانیم اطلاعات بیشتری را در حافظه اصلی نگه داریم و در نتیجه، پاسخ به جستجوها سریع‌تر انجام شود.

۲. **انتقال سریع‌تر داده از دیسک به حافظه:** الگوریتم‌های بازفشرده‌سازی مدرن آنقدر سریع هستند که زمان انتقال داده فشرده از دیسک و سپس بازفشرده‌سازی آن، معمولاً کمتر از زمان انتقال همان حجم داده به صورت فشرده‌نشده است. برای مثال، در یک سیستم جستجوی علمی مانند PubMed، بارگذاری لیست ارسال فشرده برای یک اصطلاح پزشکی رایج و سپس بازفشرده‌سازی آن، سریع‌تر از بارگذاری لیست ارسال فشرده‌نشده است.

🎨 ویژگی‌های آماری واژه‌ها: رازهای آمار در خدمت فشرده‌سازی

برای فهم بهتر فشرده‌سازی، ابتدا باید با ویژگی‌های آماری واژه‌ها آشنا شویم. در مجموعه‌های متنی بزرگ، توزیع واژه‌ها از الگوهای خاصی پیروی می‌کند که می‌توانیم از آن‌ها برای فشرده‌سازی بهتر استفاده کنیم.

جدول زیر تأثیر پیش‌پردازش بر تعداد واژه‌ها و لیست‌های ارسال را در مجموعه Reuters-RCV1 نشان می‌دهد:

- حذف اعداد: کاهش ۲% در واژگان و ۸% در لیست‌های ارسال
- تبدیل حروف به حالت یکسان (case folding): کاهش ۱۷% در واژگان
- حذف ۳۰ واژه رایج: کاهش ۱۴% در لیست‌های ارسال
- حذف ۱۵۰ واژه رایج: کاهش ۳۰% در لیست‌های ارسال
- ریشه‌یابی (stemming): کاهش ۱۷% دیگر در واژگان

نکته جالب این است که حذف واژه‌های رایج باعث کاهش چشمگیر در تعداد postings می‌شود، اما تأثیر زیادی بر اندازه ایندکس فشرده ندارد، چون لیست‌های ارسال این واژه‌ها پس از فشرده‌سازی بسیار کوچک می‌شوند.

🔗 فشرده‌سازی بدون اتلاف در مقابل فشرده‌سازی با اتلاف

در این فصل، ما بر روی فشرده‌سازی بدون اتلاف (lossless compression) تمرکز می‌کنیم، جایی که تمام اطلاعات حفظ می‌شود. اما فشرده‌سازی با اتلاف (lossy compression) نیز کاربردهای مهمی دارد:

- **فشرده‌سازی بدون اتلاف:** تمام اطلاعات حفظ می‌شود. تکنیک‌هایی که در این فصل بررسی می‌شوند از این نوع هستند.
- **فشرده‌سازی با اتلاف:** بخشی از اطلاعات حذف می‌شود. تبدیل حروف به حالت یکسان، ریشه‌یابی، و حذف واژه‌های توقف نمونه‌هایی از این نوع هستند. همچنین مدل فضای برداری (فصل ۶) و تکنیک‌های کاهش ابعاد مانند نمایه‌سازی معنایی پنهان (فصل ۱۸) نیز نمونه‌هایی از فشرده‌سازی با اتلاف هستند.

در جستجوی وب، چون کاربران معمولاً فقط نتایج اول را می‌بینند، می‌توانیم از فشرده‌سازی با اتلاف استفاده کنیم بدون اینکه تأثیر منفی بر تجربه کاربر داشته باشیم.

📊 قانون هپس: چرا واژگان مجموعه‌های متنی آنقدر بزرگ هستند؟

شاید فکر کنید زبان‌ها واژگان محدودی دارند. برای مثال، دومین نسخه دیکشنری آکسفورد بیش از ۶۰۰۰۰۰ کلمه تعریف می‌کند. اما واژگان اکثر مجموعه‌های بزرگ بسیار بزرگ‌تر از دیکشنری آکسفورد است. چرا؟

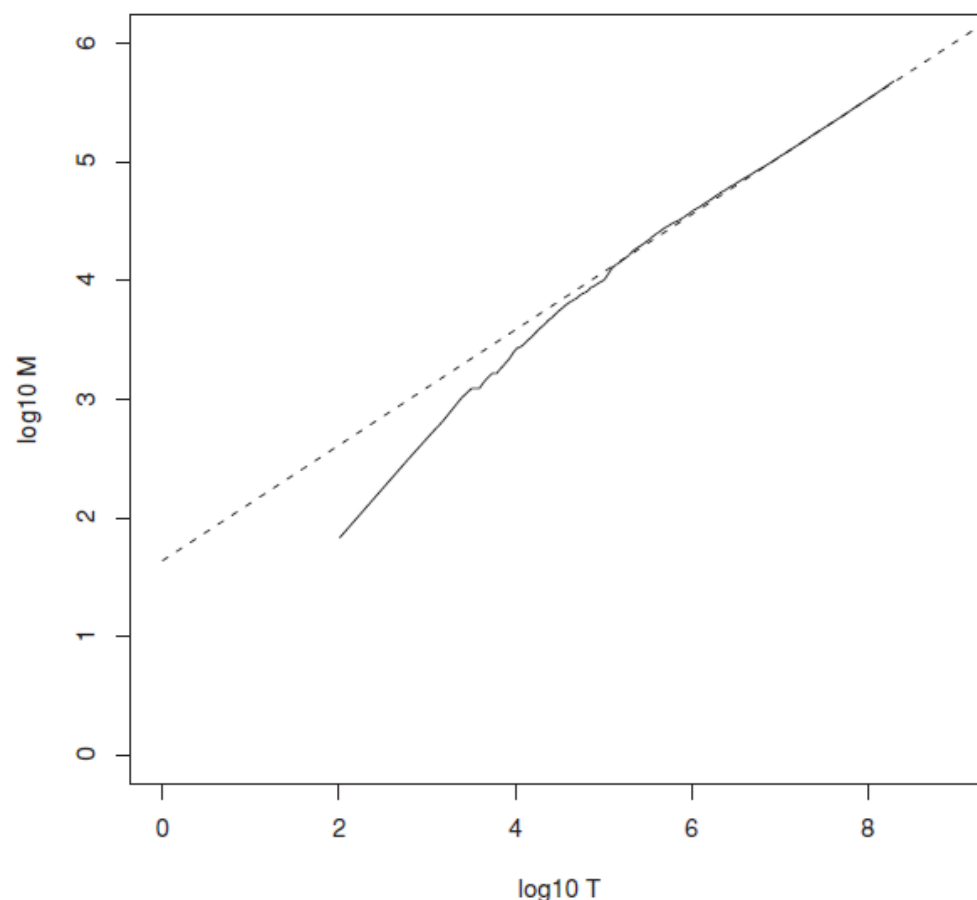
دیکشنری آکسفورد شامل اکثر نام افراد، مکان‌ها، محصولات یا موجودات علمی مانند ژن‌ها نمی‌شود. این نام‌ها باید در ایندکس معکوس گنجانده شوند تا کاربران بتوانند آن‌ها را جستجو کنند.

قانون هپس (Heaps' law) رابطه بین اندازه مجموعه (تعداد توکن‌ها) و اندازه واژگان (تعداد واژه‌های منحصره‌فرد) را توصیف می‌کند. طبق این قانون، اندازه واژگان با افزایش اندازه مجموعه به صورت زیر رشد می‌کند:

$$M = k * T^b$$

که در آن M تعداد واژه‌های منحصره‌فرد، T تعداد کل توکن‌ها، k و b ثابت‌هایی هستند که معمولاً k بین ۱۰ تا ۱۰۰ و b بین ۰.۴ تا ۰.۶ هستند.

برای مثال، در مجموعه Reuters-RCV1، بهترین برازش برای این رابطه $k = 44$ و $b = 0.49$ است. این یعنی با دو برابر کردن اندازه مجموعه، تعداد واژه‌های منحصره‌فرد حدود ۴۰٪ افزایش می‌یابد، نه دو برابر!



► Figure 5.1 Heaps' law. Vocabulary size M as a function of collection size T (number of tokens) for Reuters-RCV1. For these data, the dashed line $\log_{10} M = 0.49 * \log_{10} T + 1.64$ is the best least-squares fit. Thus, $k = 10^{1.64} \approx 44$ and $b = 0.49$.

شکل 5.1 – قانون هیپس. واژگان به اندازه M و تابعی از مجموعه ای به اندازه T (تعداد توکن ها) برای Reuters-RCV1. برای این داده ها ، خط نقطه چین بهترین کوچکترین توان مناسب است.

در ادامه این فصل، با تکنیک‌های فشرده‌سازی دیکشنری (روش دیکشنری به صورت رشته و ذخیره‌سازی بلوکی) و فشرده‌سازی لیست‌های ارسال (کدگذاری بایت متغیر و کدگذاری گاما) آشنا خواهیم شد. این تکنیک‌ها به ما کمک می‌کنند تا سیستم‌های بازیابی اطلاعات کارآمدتر و سریع‌تری بسازیم.

۵.۱.۱ قانون هیپس: چرا واژگان اینقدر سریع رشد می‌کنند؟

🌐 یک مثال از دنیای واقعی: رشد واژگان در اینستاگرام

تصور کنید مدیر داده در اینستاگرام هستید و می‌خواهید پیش‌بینی کنید چقدر فضا برای ذخیره هشتک‌های جدید نیاز دارید. در ابتدا، اینستاگرام فقط چند هزار هشتک داشت، اما امروزه میلیون‌ها هشتک منحصربه‌فرد وجود دارد. قانون هیپس به ما کمک می‌کند این رشد را پیش‌بینی کنیم!

برای تخمین تعداد واژه‌های متمایز M در یک مجموعه متنی با T توکن، از **قانون هیپس** استفاده می‌شود. این قانون رابطه‌ای تجربی بین اندازه مجموعه و اندازه واژگان ارائه می‌دهد:

$$M = k \cdot T^b$$

که در آن:

• T : تعداد توکن‌ها در مجموعه

- M : تعداد واژه‌های متمایز
- k : ضریب وابسته به نوع مجموعه (بین ۳۰ تا ۱۰۰)
- b : نمایی که معمولاً حدود ۰٫۵ است

چرا این قانون مهم است؟ 🔍

قانون هیپس نشان می‌دهد که رشد واژگان با افزایش حجم متن متوقف نمی‌شود. این یعنی برای سیستم‌های بزرگی مانند گوگل یا اینستاگرام، همیشه باید برای واژگان جدید فضا در نظر گرفت. این دقیقاً همان چیزی است که فشرده‌سازی دیکشنری را برای سیستم‌های بازیابی اطلاعات حیاتی می‌کند.

در مجموعه Reuters-RCV1، برازش خطی زیر به‌دست آمده:

$$\log_{10} M = 0.49 \cdot \log_{10} T + 1.64$$

که معادل است با:

$$M = 44 \cdot T^{0.49}$$

مثال عددی: برای $T = 1,000,020$ توکن، قانون هیپس پیش‌بینی می‌کند:

$$M = 44 \cdot (1,000,020)^{0.49} \approx 38,323$$

در حالی که مقدار واقعی برابر با 38,365 واژه بوده — که بسیار نزدیک به پیش‌بینی است.

عوامل مؤثر بر پارامترها 🧩

مقدار k وابسته به نوع مجموعه و نحوه پیش‌پردازش آن است:

- استفاده از case folding و stemming باعث کاهش نرخ رشد واژگان می‌شود.
- وجود اعداد، اشتباهات املائی و اسامی خاص باعث افزایش واژگان می‌شود.

نتیجه‌گیری مهم:

- اندازه واژگان با افزایش اسناد همچنان رشد می‌کند و به سقف نمی‌رسد.
- برای مجموعه‌های بزرگ، اندازه دیکشنری بسیار زیاد خواهد بود.
- بنابراین، فشرده‌سازی دیکشنری برای کارایی سیستم بازیابی اطلاعات ضروری است.

۵.۱.۲ قانون زیپف: چرا برخی کلمات بسیار بیشتر از بقیه تکرار می‌شوند؟ 📊

📌 یک مثال از دنیای واقعی: تحلیل توییت‌های توییتر

تصور کنید محققان توییتر (X) می‌خواهند بفهمند کلمات در توییت‌های فارسی چگونه توزیع شده‌اند. آنها متوجه می‌شوند که کلمه «از» در میلیون‌ها توییت ظاهر می‌شود، در حالی که کلمه «الگوریتم» بسیار نادر است. این توزیع ناهمگون چیزی است که قانون زیپف توصیف می‌کند!

برای طراحی الگوریتم‌های فشرده‌سازی لیست‌های ارسال، باید بدانیم واژه‌ها چگونه در مجموعه اسناد توزیع شده‌اند. یکی از مدل‌های رایج برای این هدف، **قانون زیپف** است.

این قانون بیان می‌کند که اگر واژه t_1 رایج‌ترین واژه باشد، t_2 دومین واژه رایج، و همین‌طور تا t_i ، آنگاه فراوانی مجموعه‌ای (collection frequency) واژه t_i به‌صورت زیر است:

$$cf_i \propto \frac{1}{i}$$

به زبان ساده:

- اگر رایج‌ترین واژه cf_1 بار ظاهر شود
- واژه دوم تقریباً $cf_1/2$ بار
- واژه سوم تقریباً $cf_1/3$ بار
- و همین‌طور الگو ادامه می‌یابد...

اهمیت عملی قانون زیپف 🔍

در تحلیل داده‌های توییتر، محققان متوجه شدند کلماتی مانند «از»، «به» و «در» تقریباً در هر توییت ظاهر می‌شوند، در حالی که کلمات تخصصی مانند «الگوریتم» یا «فشرده‌سازی» بسیار نادر هستند. این توزیع ناهمگون به ما کمک می‌کند تا الگوریتم‌های فشرده‌سازی بهینه طراحی کنیم.

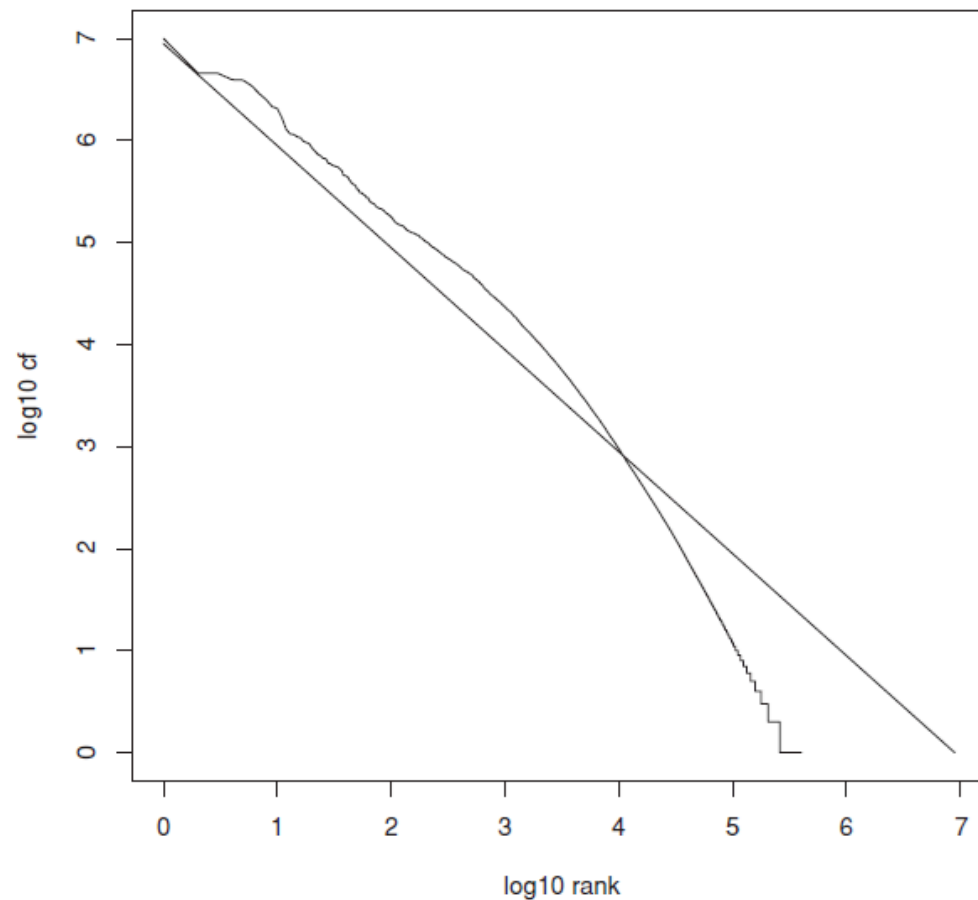
فرمول زیپف را می‌توان به‌صورت نمایی یا لگاریتمی نیز نوشت:

$$cf_i = c \cdot i^k \quad \text{with } k = -1$$

یا به‌صورت لگاریتمی:

$$\log cf_i = \log c + k \cdot \log i$$

که در آن c یک ثابت است و $k = -1$. این رابطه یک **قانون توانی (power law)** است که در بسیاری از پدیده‌های طبیعی و اجتماعی مشاهده می‌شود.



► **Figure 5.2** Zipf's law for Reuters-RCV1. Frequency is plotted as a function of frequency rank for the terms in the collection. The line is the distribution predicted by Zipf's law (weighted least-squares fit; intercept is 6.95).

نمودار بالا برای مجموعه **Reuters-RCV1**، فراوانی واژه‌ها را بر حسب رتبه‌شان نشان می‌دهد. خط با شیب 1- تقریب خوبی از داده‌ها ارائه می‌دهد، هرچند کاملاً دقیق نیست.

🧩 کاربرد در فشرده‌سازی

قانون زیپف به ما می‌گوید که:

- بیشتر واژه‌ها بسیار کم‌تکرار هستند
- فشرده‌سازی لیست‌های ارسال برای واژه‌های رایج بسیار مؤثر است
- می‌توانیم الگوریتم‌های متفاوتی برای واژه‌های پرتکرار و کم‌تکرار طراحی کنیم

در بخش 5.3، خواهیم دید که چگونه این درک از توزیع واژه‌ها به طراحی الگوریتم‌های فشرده‌سازی بهینه کمک می‌کند.

۵.۲ فشرده‌سازی دیکشنری: چرا حتی بخش کوچک هم مهم است؟ 📖

🛒 یک مثال از دنیای واقعی: جستجوی محصولات در آمازون

تصور کنید مدیر موتور جستجوی آمازون هستید. هر ثانیه، هزاران کاربر در حال جستجوی محصولات هستند. وقتی کاربری عبارت "لپ‌تاپ گیمینگ" را جستجو می‌کند، سیستم باید فوراً واژه‌های "لپ‌تاپ" و "گیمینگ" را در دیکشنری پیدا کند. اگر دیکشنری در حافظه اصلی نباشد، هر جستجو نیازمند چندین دسترسی به دیسک خواهد

بود که باعث تأخیر قابل توجهی می‌شود. در دنیای رقابتی تجارت الکترونیک، حتی تأخیر ۱۰۰ میلی‌ثانیه‌ای می‌تواند منجر به از دست رفتن مشتری شود!

در نگاه اول، دیکشنری نسبت به فایل لیست‌های ارسال (postings file) بسیار کوچک‌تر است. پس چرا باید آن را فشرده کنیم؟ پاسخ در **کارایی سیستم بازیابی اطلاعات** نهفته است.

یکی از عوامل اصلی در زمان پاسخ‌دهی سیستم IR، تعداد **disk seek**‌هایی است که برای پردازش یک پرسمان انجام می‌شود. اگر بخشی از دیکشنری روی دیسک باشد، هر جست‌وجو نیازمند چندین seek خواهد بود. بنابراین، هدف اصلی فشرده‌سازی دیکشنری این است که بتوان آن را به‌طور کامل (یا حداقل بخش بزرگی از آن) در حافظهٔ اصلی نگه داشت.

🚀 مزایای فشرده‌سازی دیکشنری در دنیای واقعی

در شرکت‌های بزرگی مانند گوگل یا مایکروسافت که باید میلیاردها سند را ایندکس کنند، فشرده‌سازی دیکشنری به آنها اجازه می‌دهد:

- کاهش تعداد disk seek در زمان جست‌وجو (هر seek معادل چند میلی‌ثانیه تأخیر است)
- افزایش throughput سیستم در پردازش پرسمان‌ها (مهم برای سرویس‌هایی با ترافیک بالا)
- امکان اجرای سیستم روی سخت‌افزارهای محدود (مثل موبایل یا کامپیوترهای جاسازی‌شده در خودروها)
- کاهش زمان راه‌اندازی سیستم (مهم برای سرویس‌های ابری که باید سریعاً راه‌اندازی شوند)
- هم‌زیستی بهتر با نرم‌افزارهای دیگر در حافظه (در محیط‌های اشتراکی)

شکل زیر یک ساختار ساده برای ذخیره‌سازی دیکشنری را نشان می‌دهد:

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

در این ساختار، هر ورودی به‌صورت **fixed-width entry** ذخیره می‌شود. به‌طور مثال:

- term: 20 bytes
- document frequency: 4 bytes
- pointer to postings list: 4 bytes

🏆 چالش در دنیای واقعی

برای یک شرکت بزرگ بین‌المللی که اسناد را به زبان‌های مختلف ایندکس می‌کند، دیکشنری می‌تواند به راحتی از حافظه یک کامپیوتر استاندارد فراتر رود. در چنین مواردی، فشرده‌سازی دیکشنری نه یک انتخاب، بلکه یک

ضرورت فنی است. در ادامه این بخش، با روش‌های مختلف فشرده‌سازی دیکشنری آشنا می‌شویم که به ما کمک می‌کنند تا حتی در محیط‌های محدود نیز سیستم‌های جستجوی کارآمد داشته باشیم.

۵.۲.۱ دیکشنری به صورت رشته‌ای: هوشمندانه‌تر استفاده از فضا 🧰

🔍 یک مثال از دنیای واقعی: بهینه‌سازی جستجو در اسناد پزشکی

تصور کنید در شرکت IBM Watson Health کار می‌کنید و در حال طراحی سیستم جستجوی پزشکی هستید که به پزشکان کمک می‌کند در میلیون‌ها مقاله پزشکی جستجو کنند. در این حوزه، واژه‌های تخصصی و طولانی مانند "hydrochlorofluorocarbons" یا "immunohistochemistry" بسیار رایج هستند. اگر از روش با طول ثابت استفاده کنید، بسیاری از این واژه‌ها کوتاه نمی‌آیند و فضای زیادی هدر می‌رود. اینجاست که روش رشته‌ای به کمک شما می‌آید!

ساده‌ترین ساختار برای ذخیره‌سازی دیکشنری، استفاده از آرایه‌ای از ورودی‌های با طول ثابت (fixed-width) است. در این روش، واژگان به صورت مرتب‌شده الفبایی ذخیره می‌شوند و هر ورودی شامل موارد زیر است:

- term: 20 bytes
- document frequency: 4 bytes
- pointer to postings list: 4 bytes

برای مجموعه **Reuters-RCV1** با 400,000 واژه، حافظه موردنیاز برابر است با:

$$400,000 \times (20 + 4 + 4) = 11.2 \text{ MB}$$

⚠️ مشکلات روش با طول ثابت

این روش بسیار ناکارآمد است، چون:

- طول متوسط واژه‌ها در انگلیسی حدود ۸ کاراکتر است ← ۱۲ بایت هدر در هر ورودی
- واژه‌های طولانی‌تر از ۲۰ کاراکتر قابل ذخیره‌سازی نیستند

در سیستم جستجوی پزشکی Watson، این مشکل جدی‌تر است، چون بسیاری از واژه‌های پزشکی بسیار طولانی هستند و نمی‌توانند در ۲۰ کاراکتر جای بگیرند!

راه‌حل: ذخیره‌سازی واژه‌ها به صورت یک رشته طولانی از کاراکترها (dictionary-as-a-string). در این روش:

- همه واژه‌ها پشت سر هم در یک رشته ذخیره می‌شوند
- هر واژه با یک **term pointer** مشخص می‌شود (آدرس شروع واژه)
- جست‌وجو با **binary search** روی جدول اشاره‌گرها انجام می‌شود

برای 400,000 واژه با طول متوسط ۸ کاراکتر، تعداد موقعیت‌های ممکن برابر است با:

$$400,000 \times 8 = 3.2 \times 10^6$$

برای آدرس‌دهی این تعداد موقعیت، به اشاره‌گرهایی با طول زیر نیاز داریم:

$$\log_2(3.2 \times 10^6) \approx 22 \text{ bits} \Rightarrow 3 \text{ bytes}$$

در این ساختار جدید، حافظه موردنیاز برابر است با:

- term: 8 bytes (میانگین)
- term pointer: 3 bytes
- document frequency: 4 bytes
- postings pointer: 4 bytes

$$400,000 \times (8 + 3 + 4 + 4) = 7.6 \text{ MB}$$

بنابراین، با استفاده از dictionary-as-a-string حدود ۳۳% صرفه‌جویی در حافظه نسبت به روش fixed-width خواهیم داشت.

نحوه کارکرد این روش 🎨

در روش dictionary-as-a-string، واژه‌ها به صورت یک رشته پیوسته ذخیره می‌شوند و اشاره‌گرها ابتدای هر واژه را مشخص می‌کنند. برای مثال، اگر سه واژه "syzygetic"، "systile" و "syzygial" را داشته باشیم، در رشته به این شکل ذخیره می‌شوند:

```
...s y s t i l e s y z y g e t i c s y z y g i a l...
```

اشاره‌گر اول به ابتدای "systile"، اشاره‌گر دوم به ابتدای "syzygetic" و اشاره‌گر سوم به ابتدای "syzygial" اشاره می‌کند. این ساختار به ما اجازه می‌دهد تا با جستجوی دودویی در جدول اشاره‌گرها، واژه مورد نظر را به سرعت پیدا کنیم.

۵.۲.۲ فشرده‌سازی بلوکی و کدگذاری پیشوندی 📦

📱 یک مثال از دنیای واقعی: بهینه‌سازی جستجو در اپلیکیشن موبایل

تصور کنید در تیم توسعه اپلیکیشن Spotify کار می‌کنید و مسئول بهینه‌سازی جستجوی آهنگ‌ها در نسخه موبایل هستید. اپلیکیشن شما باید بتواند نام صدها هزار هنرمند، آلبوم و آهنگ را در حافظه محدود موبایل ذخیره کند و همزمان جستجوی سریعی را ارائه دهد. فشرده‌سازی بلوکی و کدگذاری پیشوندی به شما اجازه می‌دهد تا دیکشنری بزرگ را در حافظه محدود موبایل جا دهید و همزمان سرعت جستجو را قابل قبول نگه دارید.

برای کاهش بیشتر اندازه دیکشنری، واژه‌ها را به بلوک‌هایی با اندازه k تقسیم می‌کنیم و فقط برای واژه اول هر بلوک یک اشاره‌گر ذخیره می‌کنیم. طول هر واژه نیز به صورت یک بایت در ابتدای آن ذخیره می‌شود.

برای $k = 4$:

- صرفه‌جویی در اشاره‌گرها: $9 \times (k - 1) = 9 \times 3$ bytes
- هزینه اضافه برای طول واژه‌ها: $k = 4$ bytes
- صرفه‌جویی خالص: 5 bytes per block
- تعداد بلوک‌ها: $400,000 / 4 = 100,000$
- صرفه‌جویی کل: $100,000 \times 5 = 0.5 \text{ MB}$

حافظه نهایی: $7.6 - 0.5 = 7.1 \text{ MB}$

🔗 معامله بین فضا و سرعت

افزایش k باعث کاهش بیشتر حافظه می‌شود، اما زمان جست‌وجو افزایش می‌یابد. در اپلیکیشن Spotify، این معامله بسیار مهم است:

- جست‌وجو در دیکشنری بدون فشرده‌سازی: میانگین 1.6 مرحله
- جست‌وجو با $k = 4$: میانگین 2 مرحله ← افزایش 25%
- جست‌وجو با $k = 8$ یا $k = 16$: کندتر خواهد شد و تجربه کاربری را تحت تأثیر قرار می‌دهد

برای اپلیکیشن موبایل که باید پاسخگو باشد، تیم Spotify احتمالاً از $k = 4$ استفاده می‌کند تا تعادل خوبی بین فضا و سرعت برقرار شود.

abc کدگذاری پیشوندی (Front Coding)

تکنیک **front coding** از اشتراک پیشوند بین واژه‌های متوالی استفاده می‌کند. در اپلیکیشن Spotify، بسیاری از نام هنرمندان با پیشوندهای مشترک شروع می‌شوند:

```
9The Beatles
10The Rolling Stones
8The Doors
8The Who
  ↓
3The*
6♦Beatles
12♦Rolling Stones
5♦Doors
3♦Who
```

در این روش، پیشوند مشترک با * علامت‌گذاری می‌شود و ادامه واژه‌ها با ♦ و طول مشخص می‌شوند. این تکنیک در مجموعه Reuters باعث صرفه‌جویی 1.2 MB دیگر شد.

🏗️ مقایسه نهایی ساختارهای دیکشنری

جدول زیر مقایسه‌ای از فضا مورد نیاز برای روش‌های مختلف فشرده‌سازی دیکشنری را نشان می‌دهد:

ساختار	حافظه (MB)
Fixed-width	11.2
Term pointers into string	7.6
Blocking (k = 4)	7.1
Blocking + Front coding	5.9

همانطور که می‌بینید، با ترکیب این تکنیک‌ها می‌توانیم تقریباً 50% در فضا صرفه‌جویی کنیم، که برای اپلیکیشن‌های موبایل با حافظه محدود بسیار حیاتی است.

۵.۳ فشرده‌سازی فایل لیست‌های ارسال: هنر ذخیره هوشمندانه 🧩

🔍 یک مثال از دنیای واقعی: بهینه‌سازی جستجو در گوگل

تصور کنید مهندس گوگل هستید و مسئول بهینه‌سازی ایندکس جستجو برای میلیاردها صفحه وب هستید. هر روز، میلیون‌ها صفحه جدید به ایندکس اضافه می‌شود و کاربران میلیاردها جستجو انجام می‌دهند. اگر هر لیست ارسال را بدون فشرده‌سازی ذخیره کنید، به هزاران سرور با حافظه عظیم نیاز خواهید داشت! فشرده‌سازی لیست‌های ارسال به گوگل اجازه می‌دهد تا با منابع کمتر، پاسخ‌های سریع‌تری به کاربران ارائه دهد.

در این بخش، هدف ما کاهش اندازه فایل لیست‌های ارسال (postings file) است. برای مجموعه **Reuters-RCV1**:

- تعداد اسناد: 800,000
- تعداد توکن‌ها در هر سند: 200
- طول هر توکن: 6 کاراکتر
- تعداد کل postings: 100,000,000

بنابراین:

- اندازه مجموعه: $800,000 \times 200 \times 6 = 960 \text{ MB}$
- اندازه فایل لیست‌های ارسال بدون فشرده‌سازی: $100,000,000 \times \frac{20}{8} = 250 \text{ MB}$

💡 ایده کلیدی: کدگذاری فاصله‌ها (Gap Encoding)

برای کاهش این حجم، از ایده **gap encoding** استفاده می‌کنیم. به جای ذخیره مستقیم docIDها، فاصله بین آن‌ها را ذخیره می‌کنیم:

term	docIDs	gaps
the	283045, 283044, 283043, 283042	1, 1, 1
computer	283202, 283159, 283154, 283047	43, 5, 107

	gaps	docIDs	term
	248100 ,252000	500100 ,252000	arachnocentric

برای واژه‌های رایج مانند "the"، فاصله‌ها بسیار کوچک هستند (معمولاً ۱) و می‌توان آن‌ها را با بیت‌های بسیار کمتر ذخیره کرد. اما برای واژه‌های نادر مانند "arachnocentric"، فاصله‌ها بزرگ هستند و همچنان نیاز به بیت‌های کامل دارند.

روش‌های فشرده‌سازی

برای فشرده‌سازی این فاصله‌ها، از دو نوع روش استفاده می‌کنیم:

- **Bytewise Compression**: فشرده‌سازی با حداقل تعداد بایت
- **Bitwise Compression**: فشرده‌سازی با حداقل تعداد بیت

در گوگل، هر دو روش استفاده می‌شود. برای واژه‌های بسیار رایج که فاصله‌های کوچکی دارند، روش bitwise بسیار کارآمد است، زیرا می‌تواند فاصله ۱ را تنها با یک بیت ذخیره کند. برای واژه‌های کمتر رایج، روش bitwise ممکن است مناسب‌تر باشد.

در بخش‌های بعدی، دو الگوریتم اصلی از هر دسته را بررسی و پیاده‌سازی می‌کنیم:

- Variable Byte Encoding (bytewise)
- Gamma Encoding (bitwise)

اهمیت عملی

در سیستم‌های بزرگ مانند گوگل یا آمازون، فشرده‌سازی لیست‌های ارسال فقط یک بهینه‌سازی کوچک نیست، بلکه یک ضرورت فنی است. بدون این تکنیک‌ها، هزینه ذخیره‌سازی و پردازش ایندکس‌ها به قدری بالا می‌رود که ارائه خدمات جستجوی سریع و مقرون‌به‌صرفه غیرممکن می‌شود. با فشرده‌سازی هوشمندانه، این شرکت‌ها می‌توانند میلیاردها سند را با هزینه کمتر ایندکس کرده و پاسخ‌های سریع‌تری به کاربران ارائه دهند.

۵.۳.۱ کدگذاری بایتی متغیر: هنر ذخیره هوشمندانه اعداد

یک مثال از دنیای واقعی: بهینه‌سازی جستجو در واتس‌اپ

تصور کنید در تیم واتس‌اپ کار می‌کنید و مسئول بهینه‌سازی جستجوی پیام‌ها هستید. هر روز میلیاردها پیام ارسال می‌شود و کاربران باید بتوانند سریعاً پیام‌های قدیمی خود را پیدا کنند. اگر هر شناسه پیام را با ۴ بایت ذخیره کنید، حجم ایندکس به سرعت افزایش می‌یابد. با استفاده از کدگذاری بایتی متغیر، واتس‌اپ می‌تواند شناسه‌های پیام‌های نزدیک به هم را با فضای بسیار کمتری ذخیره کند و همزمان سرعت جستجو را بالا نگه دارد.

در روش کدگذاری بایتی متغیر (Variable Byte Encoding)، هر عدد (مثلاً فاصله بین docIDها) با تعداد متغیری از بایت‌ها ذخیره می‌شود. این روش از یک ایده ساده اما هوشمندانه استفاده می‌کند:

🔍 ساختار یک بایت در کدگذاری VB

هر بایت در این روش شامل دو بخش است:

- ۷ بیت داده (payload): بخشی از عدد را ذخیره می‌کند
- ۱ بیت ادامه (continuation bit): نشان می‌دهد آیا این بایت آخرین بایت عدد است یا خیر

اگر continuation bit برابر ۱ باشد ← این بایت آخرین بایت عدد است

اگر continuation bit برابر ۰ باشد ← عدد در بایت‌های بعدی ادامه دارد

برای مثال، عدد 214577 به صورت زیر کدگذاری می‌شود:

214577 = 00001101 00001100 10110001 → 3 bytes: [13, 12, 177] → continuation bits: 0, 0, 1

📌 فرآیند کدگذاری و رمزگشایی

برای کدگذاری یک عدد:

1. عدد را به مبنای 128 می‌بریم (چون 7 بیت برای داده داریم)
2. بایت‌ها را از پایین به بالا تولید می‌کنیم
3. به آخرین بایت، 128 اضافه می‌کنیم تا continuation bit آن 1 شود

برای رمزگشایی:

1. بایت‌ها را یکی‌یکی می‌خوانیم
2. تا زمانی که continuation bit صفر است، عدد را با ضرب در 128 ادامه می‌دهیم
3. وقتی continuation bit برابر ۱ شد، عدد کامل شده است


```

VBENCODENUMBER(n)
1  bytes  $\leftarrow \langle \rangle$ 
2  while true
3  do PREPEND(bytes, n mod 128)
4    if n < 128
5      then BREAK
6    n  $\leftarrow n \div 128$ 
7  bytes[LENGTH(bytes)] += 128
8  return bytes

VBENCODE(numbers)
1  bytestream  $\leftarrow \langle \rangle$ 
2  for each n  $\in$  numbers
3  do bytes  $\leftarrow$  VBENCODENUMBER(n)
4    bytestream  $\leftarrow$  EXTEND(bytestream, bytes)
5  return bytestream

VBDECODE(bytestream)
1  numbers  $\leftarrow \langle \rangle$ 
2  n  $\leftarrow 0$ 
3  for i  $\leftarrow 1$  to LENGTH(bytestream)
4  do if bytestream[i] < 128
5    then n  $\leftarrow 128 \times n + \text{bytestream}[i]$ 
6    else n  $\leftarrow 128 \times n + (\text{bytestream}[i] - 128)$ 
7      APPEND(numbers, n)
8    n  $\leftarrow 0$ 
9  return numbers

```

► Figure 5.8 VB encoding and decoding. The functions *div* and *mod* compute integer division and remainder after integer division, respectively. PREPEND adds an element to the beginning of a list, for example, PREPEND($\langle 1, 2 \rangle$, 3) = $\langle 3, 1, 2 \rangle$. EXTEND extends a list, for example, EXTEND($\langle 1, 2 \rangle$, $\langle 3, 4 \rangle$) = $\langle 1, 2, 3, 4 \rangle$.

شکل 5.8 – کدگذاری و کدگشایی بایتی متغیر. توابع *div* و *mod* به ترتیب تقسیم صحیح و باقی مانده آن را محاسبه می کنند. Prepend یک عنصر را به ابتدای لیست اضافه می کند. به عنوان مثال: Prepend($\langle 1, 2 \rangle$, 3) = $\langle 3, 1, 2 \rangle$. Extend یک لیست را بسط می دهد. به عنوان مثال: Extend($\langle 1, 2 \rangle$, $\langle 3, 4 \rangle$) = $\langle 1, 2, 3, 4 \rangle$.

در آزمایش روی مجموعه Reuters-RCV1، اندازه فایل postings از 250MB به 116MB کاهش یافت – یعنی بیش از ۵۰٪ صرفه جویی در فضا.

مزایا و معایب

کدگذاری VB تعادلی عالی بین سرعت و فشرده سازی ارائه می دهد:

- **مزیت:** پیاده سازی ساده، سرعت رمزگشایی بالا، و فشرده سازی قابل قبول
- **معایب:** برای اعداد بسیار بزرگ (مانند فاصله های بزرگ در واژه های نادر) کارایی کمتری دارد

برای اکثر سیستم های بازیابی اطلاعات، این روش یک انتخاب عالی است. اما اگر فضا بسیار محدود است، می توان از روش های سطح بیت (مانند γ codes) که در بخش بعدی معرفی می شوند، استفاده کرد.

```
In [5]: def vb_encode_number(n):
        """
        این تابع یک عدد را با استفاده از روش کدگذاری بایتی متغیر کدگذاری می کند.
        است (continuation bit) هر بایت شامل 7 بیت داده و 1 بیت ادامه
```



```
. (MSB=1) بیت ادامه برای تمام بایت‌ها به جز آخرین بایت تنظیم می‌شود.
"""

# حالت خاص برای عدد صفر، یک بایت با مقدار صفر برمی‌گرداند
if n == 0:
    return [0]

bytes_ = []
while True:
    # استخراج 7 بیت کم‌اهمیت
    byte = n & 0x7F
    n >>= 7

    # اگر هنوز بیت‌های باقی‌مانده وجود دارد، بیت ادامه را تنظیم کن
    if n > 0:
        byte |= 0x80

    bytes_.append(byte)
    if n == 0:
        break

return bytes_

# (gaps مثلاً) تابع کدگذاری لیستی از اعداد
def vb_encode(numbers):
    """
    این تابع لیستی از اعداد را با استفاده از روش کدگذاری بایتهای متغیر کدگذاری می‌کند.
    """
    bytestream = []
    for n in numbers:
        bytestream.extend(vb_encode_number(n))
    return bytestream

# تابع رمزگشایی لیست بایت‌ها به لیست اعداد
def vb_decode(bytestream):
    """
    این تابع لیست بایت‌های کدگذاری شده را به لیست اعداد اصلی بازمی‌گرداند.
    """
    numbers = []
    n = 0
    shift = 0 # شیفت برای قرار دادن بایت‌ها در موقعیت صحیح

    for byte in bytestream:
        # استخراج 7 بیت داده
        data = byte & 0x7F
        # قرار دادن بیت‌ها در موقعیت صحیح
        n |= (data << shift)

        # این آخرین بایت عدد است، (byte < 128) اگر بیت ادامه تنظیم نشده باشد
        if byte < 128:
            numbers.append(n)
            n = 0
            shift = 0
        else:
            # در غیر این صورت، برای بایت بعدی شیفت را 7 واحد افزایش می‌دهیم
            shift += 7

    return numbers

# تابع محاسبه حجم قبل و بعد از فشرده‌سازی
def compare_space(gaps):
    """
    ها را قبل و بعد از فشرده‌سازی مقایسه می‌کند gaps این تابع حجم لیست
    """
    # حجم قبل از فشرده‌سازی (هر عدد 4 بایت)
    original_size = len(gaps) * 4

    # حجم بعد از فشرده‌سازی
    encoded = vb_encode(gaps)
    compressed_size = len(encoded)

    return original_size, compressed_size

# مثال با داده‌های واقعی از کتاب
gaps = [5, 214577]
encoded = vb_encode(gaps)
decoded = vb_decode(encoded)

print("Gaps:", gaps)
print("Encoded bytes:", encoded)
print("Decoded:", decoded)
print()

# مقایسه حجم قبل و بعد از فشرده‌سازی
original_size, compressed_size = compare_space(gaps)
print(f"Original size: {original_size} bytes")
print(f"Compressed size: {compressed_size} bytes")
print(f"Compression ratio: {compressed_size/original_size:.2f}")

# gaps مثال با لیست بزرگتر از
large_gaps = [1, 1, 1, 1, 1, 107, 5, 43, 252000, 248100]
original_size, compressed_size = compare_space(large_gaps)
print()
print("Large gaps example:")
print(f"Original size: {original_size} bytes")
print(f"Compressed size: {compressed_size} bytes")
print(f"Compression ratio: {compressed_size/original_size:.2f}")
```

Large gaps example:
Original size: 40 bytes
Compressed size: 14 bytes
Compression ratio: 0.35

- [illegible]

1. ابتدا unary code تا رسیدن به ۰ می‌خوانیم (۱۵ بار ۱ و سپس ۰) ← طول $\text{offset} = ۱۵$ بیت
2. سپس ۱۵ بیت بعدی را می‌خوانیم (offset)
3. یک ۱ به ابتدای offset اضافه می‌کنیم ← عدد اصلی

ویژگی‌های برجسته کدهای گاما

کدهای گاما سه ویژگی کلیدی دارند که آنها را برای فشرده‌سازی ایندکس ایده‌آل می‌کند:

- **Universal:** برای هر توزیع احتمالاتی، نزدیک به کد بهینه است. این یعنی بدون اینکه توزیع فاصله‌ها را بدانیم، می‌توانیم از این کد استفاده کنیم.
- **Prefix-free:** هیچ کد گاما پیشوند دیگری نیست. این ویژگی رمزگشایی را ساده می‌کند، چون نیازی به جداکننده بین کدها نیست.
- **Parameter-free:** نیازی به تنظیم پارامتر ندارد. این ویژگی برای ایندکس‌های پویا که توزیع فاصله‌ها ممکن است تغییر کند، بسیار مهم است.

طول کد γ برای عدد G برابر است با:

$$\text{length} = 2 \cdot \lfloor \log_2 G \rfloor + 1$$

این کد همیشه طولی فرد دارد و در بدترین حالت، حداکثر دو برابر حد پایین اطلاعاتی است.

کارایی در دنیای واقعی

در مجموعه Reuters-RCV1، کدگذاری گاما باعث کاهش اندازه فایل postings از ۲۵۰MB به ۱۰۱MB شد – حدود ۱۵% بهتر از Variable Byte Encoding. این صرفه‌جویی در مقیاس وب، که گوگل باید میلیاردها صفحه را ایندکس کند، به ترابایت‌ها فضا می‌رسد!

برای تخمین اندازه ایندکس فشرده‌شده با γ ، از مدل Zipf و فرمول زیر استفاده می‌شود:

$$\text{total bits} \approx \sum_{j=1}^{M/L_c} 2NL_c \cdot \frac{\log_2 j}{j}$$

که برای $M = 400,000$ و $L_c = 15$ تقریباً برابر با ۲۲۴MB می‌شود – اما مقدار واقعی ۱۰۱MB است چون فرض‌های مدل ساده‌سازی شده‌اند.

کدهای گاما تعادل خوبی بین فشرده‌سازی و سرعت رمزگشایی ارائه می‌دهند. اگرچه رمزگشایی آنها از Variable Byte Encoding کندتر است، اما فشرده‌سازی بهتری ارائه می‌دهند. انتخاب بین این دو روش به نیازهای کاربرد بستگی دارد: اگر فضا محدود است، γ codes انتخاب بهتری هستند؛ اگر سرعت پردازش پرمهم‌تر است، Variable Byte Encoding ممکن است مناسب‌تر باشد.

```
"""
این تابع یک عدد صحیح را با استفاده از روش کدگذاری گاما کدگذاری می‌کند.
فرآیند به این صورت است:
1. عدد را به فرم دودویی تبدیل می‌کنیم.
2. (offset) بیت اول را حذف کرده.
3. (length) کدگذاری می‌کنیم unary را محاسبه کرده و به صورت offset طول.
4. length و offset را به هم متصل می‌کنیم.
"""

if n <= 0:
    raise ValueError("کدگذاری گاما فقط برای اعداد صحیح مثبت پشتیبانی می‌شود")

# مرحله 1: تبدیل به دودویی و حذف پیشوند 0'
binary = bin(n)[2:]

# مرحله 2: حذف بیت اول برای ساخت offset
offset = binary[1:]

# Length برای unary و ساخت کد offset مرحله 3: محاسبه طول
length = '1' * len(offset) + '0'

# offset و Length مرحله 4: اتصال
return length + offset

# تابع رمزگشایی یک کد گاما
def gamma_decode(code):
    """
    این تابع یک کد گاما را به عدد اصلی بازمی‌گرداند.
    فرآیند به این صورت است:
    1. (offset طول) تعداد 1های اولیه را می‌شمارد تا به 0 برسیم.
    2. را می‌خوانیم offset به تعداد بیت.
    3. اضافه کرده و به دودویی تبدیل می‌کنیم offset یک 1 به ابتدای.
    """

    # مرحله 1: شمارش تعداد 1ها برای پیدا کردن طول
    length = 0
    i = 0
    while i < len(code) and code[i] == '1':
        length += 1
        i += 1

    # مرحله 2: خواندن offset
    offset = code[i+1:i+1+length]

    # مرحله 3: بازسازی عدد اصلی
    return int('1' + offset, 2)

# تابع رمزگشایی یک جریان از کدهای گاما
def gamma_decode_stream(bit_stream):
    """
    این تابع یک جریان از بیت‌ها را که شامل چندین کد گاما است را رمزگشایی می‌کند.
    این تابع برای نمایش نحوه کار با لیست‌های ارسال فشرده‌شده مفید است.
    """

    numbers = []
    i = 0
    while i < len(bit_stream):
        # پیدا کردن طول کد بعدی
        length = 0
        while i < len(bit_stream) and bit_stream[i] == '1':
            length += 1
            i += 1

        if i >= len(bit_stream):
            break

        # خواندن offset
        offset = bit_stream[i+1:i+1+length]

        # بازسازی عدد و اضافه به لیست
        numbers.append(int('1' + offset, 2))

        # حرکت به ابتدای کد بعدی
        i += 1 + length

    return numbers

# تست توابع با اعداد نمونه از کتاب و مثال‌های واقعی
test_numbers = [1, 2, 3, 4, 9, 13, 24, 511, 1025, 5, 214577]

print("تست کدگذاری و رمزگشایی")
print("-" * 60)
print(f"{{رمزگشایی:'10'}} | {{طول کد:'10'}} | {{کد گاما:'25'}} | {{عدد:'10'}}")
print("-" * 60)

for n in test_numbers:
    encoded = gamma_encode(n)
    decoded = gamma_decode(encoded)
    print(f"{{n:<10}} | {{encoded:<25}} | {{len(encoded):<10}} | {{decoded:<10}}")

print("\n\nتست رمزگشایی جریان بیت‌ها")
print("-" * 60)
# ایجاد یک جریان از کدهای گاما برای تست
stream = ''.join([gamma_encode(n) for n in test_numbers])
decoded_stream = gamma_decode_stream(stream)
print(f"جریان بیت‌ها: {stream[:50]}...")
print(f"اعداد رمزگشایی شده: {decoded_stream}")

# محاسبه و نمایش طول کد برای چند عدد نمونه
print("\n\nتحلیل طول کدها")
```

```
print("-" * 60)
print(f"{'10>:عدد'} | {'15>:گاما'} | {'2*floor(log2(n))+1':<15}")
print("-" * 60)

for n in [1, 2, 3, 4, 9, 13, 24, 511, 1025]:
    encoded = gamma_encode(n)
    theoretical_length = 2 * (len(bin(n)) - 3) + 1
    print(f"{n:<10} | {len(encoded):<15} | {theoretical_length:<15}")
```

تست کدگذاری و رمزگشایی:

عدد		کد گاما		طول کد		رمزگشایی
1		1		0		1
2		3		100		2
3		3		101		3
4		5		11000		4
9		7		1110001		9
13		7		1110101		13
24		9		111101000		24
511		17		1111111011111111		511
1025		21		1111111110000000001		1025
5		5		11001		5
214577		35		1111111111111111010100011000110001		214577

تست رمزگشایی جریان بیت‌ها:

...جریان بیت‌ها: 01001011110001110001111010111110100011111110111111
اعداد رمزگشایی شده: [1, 2, 3, 4, 9, 13, 24, 511, 1025, 5, 214577]

تحلیل طول کدها:

عدد		2*floor(log2(n))+1		طول کد گاما		2
1		1		1		1
2		3		3		2
3		3		3		3
4		5		5		4
9		7		7		9
13		7		7		13
24		9		9		24
511		17		17		511
1025		21		21		1025

جمع‌بندی فصل ۵: فشرده‌سازی ایندکس 📌

در این فصل:

- دلایل فنی و عملی فشرده‌سازی ایندکس – از کاهش فضای دیسک تا افزایش سرعت جست‌وجو – را بررسی کردیم.
- ویژگی‌های آماری واژه‌ها را با قانون‌های Heaps و Zipf مدل‌سازی کردیم تا رفتار واژگان در مجموعه‌های بزرگ را تحلیل کنیم.
- روش‌های فشرده‌سازی دیکشنری را معرفی کردیم: ذخیره‌سازی رشته‌ای، بلوکی (blocking)، و کدگذاری پیشوندی (front coding).
- دو الگوریتم اصلی فشرده‌سازی لیست‌های ارسال را پیاده‌سازی کردیم: کدگذاری بایتی متغیر (Variable Byte) و کدگذاری گاما (γ).
- مقایسهٔ دقیق بین نرخ فشرده‌سازی، سرعت رمزگشایی، و پیچیدگی پیاده‌سازی این الگوریتم‌ها را انجام دادیم.

این فصل پایه‌های ****مهندسی فشرده‌سازی داده‌ها در سیستم‌های بازیابی اطلاعات**** را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- مدل‌های رتبه‌بندی و وزن‌دهی واژه‌ها (فصل ۶)
- ارزیابی کیفیت نتایج جست‌وجو (فصل ۸)
- جست‌وجوی وب و ایندکس‌سازی در مقیاس جهانی (فصل ۲۰)

آشنا خواهیم شد – که همگی بر همین زیرساخت فشرده‌سازی دقیق و قابل توسعه بنا شده‌اند.